

The ***k*-medoids algorithm** is a [clustering algorithm](#) related to the [k-means](#) algorithm and the [medoidshift](#) algorithm. Both the *k*-means and *k*-medoids algorithms are partitional (breaking the dataset up into groups) and both attempt to minimize the distance between points labeled to be in a cluster and a point designated as the center of that cluster. In contrast to the *k*-means algorithm, *k*-medoids chooses datapoints as centers ([medoids](#)) and works with a generalization of the [Manhattan Norm](#) to define distance between datapoints.

k-medoid is a classical partitioning technique of clustering, which clusters the data set of n objects into k clusters, with the number k of clusters assumed known *a priori*. If unknown, k can be determined with methods such as [silhouette](#). It is more robust to noise and outliers as compared to [k-means](#) because it minimizes a sum of pairwise dissimilarities instead of a sum of squared Euclidean distances.

A [medoid](#) can be defined as the object of a cluster whose average dissimilarity to all the objects in the cluster is minimal. i.e. it is a most centrally located point in the cluster.

Euclidean: Take the square root of the sum of the squares of the differences of the coordinates.

For example, if $x = (a, b)$ and $y = (c, d)$, the Euclidean distance between x and y is

$$\sqrt{(a - c)^2 + (b - d)^2}.$$

Manhattan: Take the sum of the absolute values of the differences of the coordinates.

For example, if $x = (a, b)$ and $y = (c, d)$, the Manhattan distance between x and y is

$$|a - c| + |b - d|.$$

For your vectors, it's the same thing except you have more coordinates.

Algorithms

The most common realisation of k -medoid clustering is the **Partitioning Around Medoids (PAM)** algorithm. PAM uses a greedy search which may not find the optimum solution, but it is faster than exhaustive search. It works as follows:

1. Initialize: select k of the n data points as the medoids
2. Associate each data point to the closest medoid.
3. While the cost of the configuration decreases:
 1. For each medoid m , for each non-medoid data point o :
 1. Swap m and o , associate each data point to the closest medoid, recompute the cost (sum of distances of points to their medoid)
 2. If the total cost of the configuration increased in the previous step, undo the swap

Algorithms other than PAM have also been suggested in the literature, including the following [Voronoi iteration](#) method.

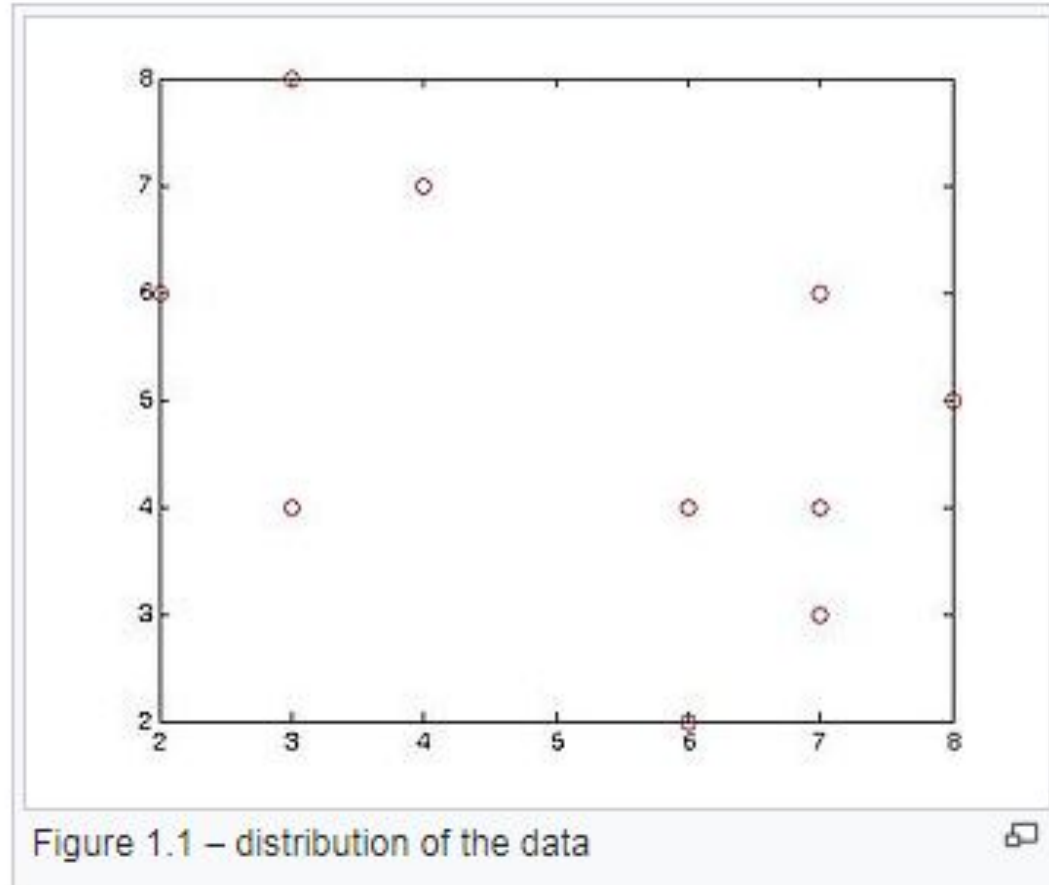
1. Select initial medoids
2. Iterate while the cost decreases:
 2. In each cluster, make the point that minimizes the sum of distances within the cluster the medoid
 3. Reassign each point to the cluster defined by the closest medoid determined in the previous step.

Example

Cluster the following data set of ten objects into two clusters i.e. $k = 2$.

Consider a data set of ten objects as follows :

X_1	2	6
X_2	3	4
X_3	3	8
X_4	4	7
X_5	6	2
X_6	6	4
X_7	7	3
X_8	7	4
X_9	8	5
X_{10}	7	6

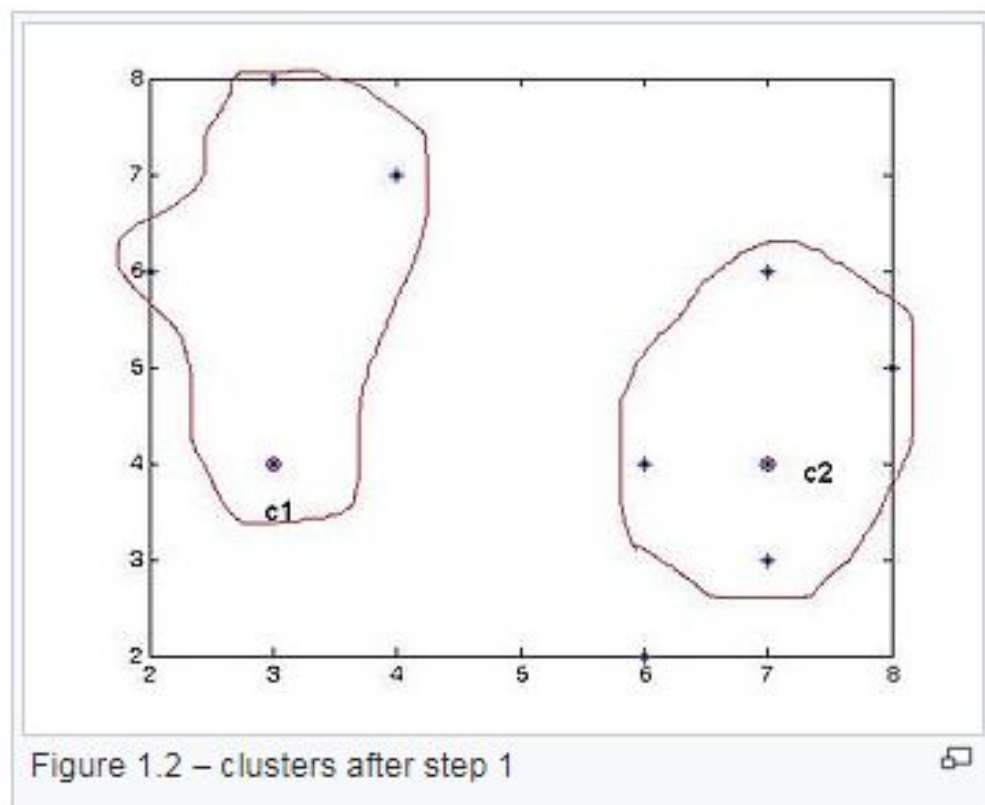


Step 1 [\[edit \]](#)

Two observations ($c_1 = x_2 = (3, 4)$ and $c_2 = x_8 = (7, 4)$) are randomly selected as medoids (cluster centers).

Manhattan distances are calculated to each center to associate each data object to its nearest medoid (marked in bold).

Data object		Distance to	
i	X_i	$c_1 = (3, 4)$	$c_2 = (7, 4)$
1	(2, 6)	3	7
2	(3, 4)	0	4
3	(3, 8)	4	8
4	(4, 7)	4	6
5	(6, 2)	5	3
6	(6, 4)	3	1
7	(7, 3)	5	1
8	(7, 4)	4	0
9	(8, 5)	6	2
10	(7, 6)	6	2
Cost		11	9



Since the points (2,6) (3,4) (3,8) and (4,7) are closer to c_1 hence they form one cluster whilst the remaining points form another cluster. Therefore, the clusters become:

$$\text{Cluster}_1 = \{(2,6)(3,4)(3,8)(4,7)\}$$

$$\text{Cluster}_2 = \{(7,4)(6,2)(6,4)(7,3)(8,5)(7,6)\}$$

The total cost of this clustering is the sum of the distance between a data point and its cluster center:

$$\underbrace{3 + 0 + 4 + 4}_{\text{objects in cluster 1}} + \underbrace{3 + 1 + 1 + 0 + 2 + 2}_{\text{objects in cluster 2}} = 20$$

Step 2 [\[edit\]](#)

Select one of the nonmedoids O'

Let us assume $O' = (7,3)$, i.e. x_7 .

So now the medoids are $c_1(3,4)$ and $O'(7,3)$

If c_1 and O' are new medoids, calculate the total cost involved

By using the formula in the step 1

i	c_1		Data objects (X_i)		Cost (distance)
1	3	4	2	6	3
3	3	4	3	8	4
4	3	4	4	7	4
5	3	4	6	2	5
6	3	4	6	4	3
8	3	4	7	4	4
9	3	4	8	5	6
10	3	4	7	6	6

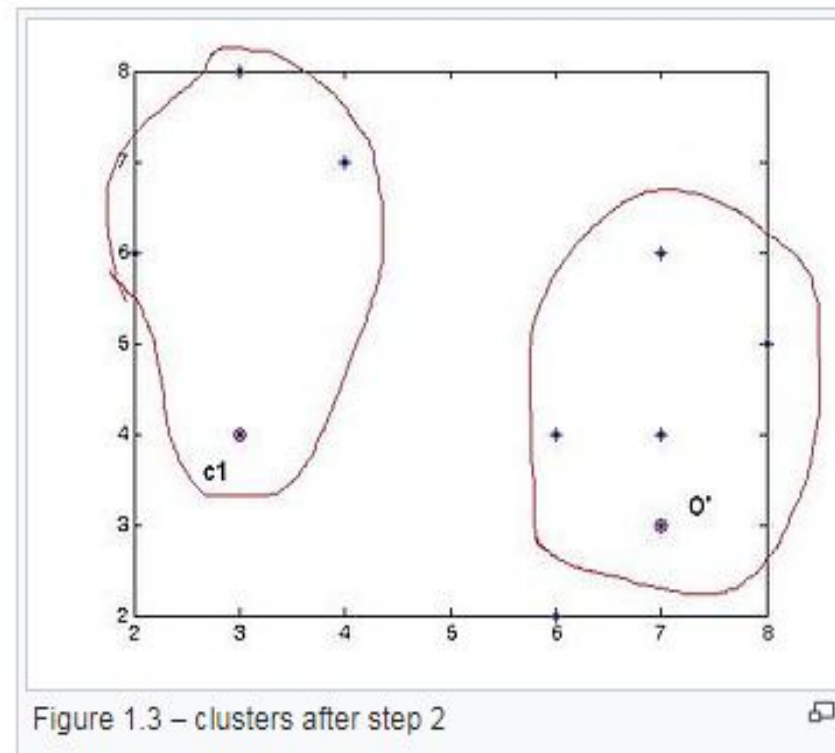


Figure 1.3 – clusters after step 2

i	O'		Data objects (X_i)		Cost (distance)
1	7	3	2	6	8
3	7	3	3	8	9
4	7	3	4	7	7
5	7	3	6	2	2
6	7	3	6	4	2
8	7	3	7	4	1
9	7	3	8	5	3
10	7	3	7	6	3

$$\begin{aligned}\text{total cost} &= 3 + 4 + 4 + 2 + 2 + 1 + 3 + 3 \\ &= 22\end{aligned}$$

So cost of swapping medoid from c_2 to O' is

$$\begin{aligned}S &= \text{current total cost} - \text{past total cost} \\ &= 22 - 20 \\ &= 2 > 0.\end{aligned}$$

So moving to O' would be a bad idea, so the previous choice was good. So we try other nonmedoids points to get minimum distance.

It may happen some data points may shift from one cluster to another cluster depending upon their closeness to medoid.

In some standard situations, k-medoids demonstrate better performance than k-means. An example is presented in Fig. 2. The most time-consuming part of the k-medoids algorithm is the calculation of the distances between objects. If a quadratic preprocessing and storage is applicable, the distances matrix can be precomputed to achieve consequent speed-up. See for example,^[3] where the authors also introduce a heuristic to choose the initial k medoids.

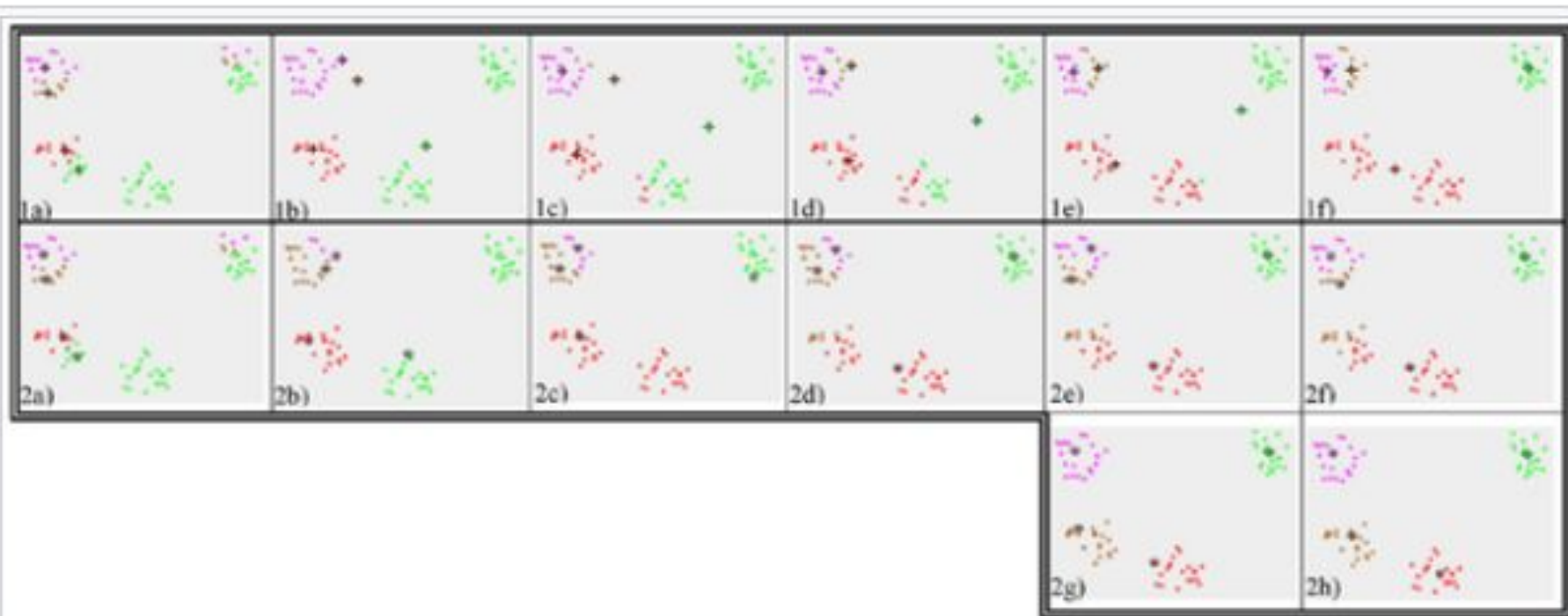


Figure 2. K-medoids versus k-means. Figs 2.1a-2.1f present a typical example of the k-means convergence to a local minimum. This result of [k-means clustering](#) contradicts the obvious cluster structure of data set. In this example, k-medoids algorithm (Figs 2.2a-2.2h) with the same initial position of medoids (Fig. 2.2a) converges to the obvious cluster structure. The small circles are data points, the four ray stars are centroids (means), the nine ray stars are medoids.^[4]